

Drawing pixels on a Keithley DMM6500 from TSP

An empirically reverse-engineered, undocumented display API: `display.setpixel`, `display.flush`, and the image-object form that makes them work.

Status	UNOFFICIAL Not part of any Keithley documentation. Everything here was found by experiment and can change or vanish in another firmware release.
Verified on	Keithley DMM6500, firmware 1.7.17a , in TSP command mode (Lua 5.0.2), driven over the raw socket on port 5025.
Evidence rule	Every API call, error string and timing number below was observed on real hardware. The few statements that are reasoning rather than observation are tagged INFERRED .
Why it matters	Pixel writes are ~7× faster than the only documented plotting path (<code>buffer.write.reading()</code> into a graph), which makes a live on-instrument plot practical.

Contents

1. What this is, and what it is not
2. Quick start — minimal working example
3. API reference
4. Failure modes and error codes
5. Measured performance
6. Animation: differential redraw
7. Practical display notes
8. Recovering a blacked-out screen
9. Things that do not exist (hallucination warning)
10. Other undocumented names in the same firmware
11. Environment gotchas

1. What this is, and what it is not

The DMM6500's TSP environment exposes a display API for building instrument-resident apps: screens, buttons, labels, lines, images. That API is documented, partly in the Reference Manual and more completely in the community "TTI Display API" reference.

`display.setpixel` **appears in neither**. It exists in the firmware and in the `display` metatable, it works, and it is the fastest way to get arbitrary graphics onto the panel from a script. This document is the result of finding out how to call it.

What you get: individual pixel writes into an off-screen buffer, plus an explicit flush that pushes the buffer to the display processor. That is the whole primitive set. There is **no** line, rectangle, circle, blit or text-into-bitmap call — see §9. If you want a line, you rasterise it yourself, one `setpixel` at a time, or you use a documented `display.OBJ_LINE` object.

READ THIS BEFORE YOU EXPERIMENT

The display setters are permissive C bindings. They accept the *wrong number of arguments* without raising a Lua error, so `pcall()` will tell you a malformed call succeeded when it did not. To know whether a call actually worked you must read `eventLog.next()` afterwards, or look at the screen.

Junk arguments to display setters can leave the physical panel completely dark. This is recoverable without a power cycle — see §8.

2. Quick start — minimal working example

Four steps: upload a PNG to act as the canvas backing store, create a screen, create an image object *with an explicit width and height*, then write pixels and flush.

2.1 Upload the canvas bitmap

`loadimage` is a **shell directive, not a Lua function**. It is intercepted by the command parser, so you send it as raw lines over the socket; you cannot call it from inside a script body or wrap it in Lua. The payload is a base64-encoded PNG, which may be split across as many lines as you like.

```
loadimage mycanvas
iVBORw0KGgoAAAANSUgAAAlgAAAH0CAIAAAD...
...more base64 lines...
endimage
```

Notes from practice:

- We sent the base64 in **900-character lines** and allowed about **one second** after `endimage` before creating the object. Neither number is a documented requirement, but uploads of a 750×500 image are large and the parser needs the time.
- The official documentation asks for "compressed 24 bit with Alpha". A plain **24-bit truecolour PNG (colour type 2, no alpha)** works fine — that is what all the tests here used, generated in a few lines of Python with `zlib`.
- Make the PNG a flat fill of your intended background colour. Its pixels are the canvas's initial contents.
- `loadimage mycanvas myscript` associates the image with a script so it survives a power cycle. Without the script name the image lives only until the next reboot, which is what you want while iterating.

2.2 Create the screen, the canvas, and draw

```
-- a screen to host the canvas
scr = display.create(display.ROOT, display.OBJ_SCREEN, "MY APP")
display.changescreen(scr)

-- the image object. The trailing WIDTH, HEIGHT are the crucial part.
im = display.create(scr, display.OBJ_IMAGE, 25, 55, "mycanvas", 750, 500)

-- write pixels: 0-based coordinates, colour is 0xRRGGBB
for x = 0, 749 do
  display.setpixel(im, x, 250, 0x00FF00)
end

-- one flush per frame pushes the whole batch to the display processor
display.flush(im)
```

THE ONE THING THAT MAKES THIS WORK

The documented `display.create()` form for an image object takes five arguments and ends at the image name:

```
display.create(<parent>, display.OBJ_IMAGE, <x>, <y>, "<image name>")
```

An image created that way *renders correctly but has no addressable pixel buffer*. Every `setpixel` into it fails. You must pass the undocumented sixth and seventh arguments:

```
display.create(<parent>, display.OBJ_IMAGE, <x>, <y>, "<name>", <width>, <height>)
```

3. API reference

3.1 `display.create()` — image object with pixel buffer **UNDOCUMENTED FORM**

```
<object id> = display.create(<parent id>, display.OBJ_IMAGE,
                             <x>, <y>, "<image name>", <width>, <height>)
```

Argument	Meaning
parent id	Screen or swipe object that owns the image. A user screen created with <code>display.OBJ_SCREEN</code> is the normal choice.
x, y	Position of the image's top-left corner within the parent, in panel pixels.
image name	The name given in the <code>loadimage</code> directive. Its bitmap becomes the canvas's initial content.
width, height	Undocumented. Size of the addressable pixel buffer. Omit these and the buffer is zero-sized: valid <code>x</code> becomes the empty range 0...-1.

Returns the object id, which is what `setpixel` and `flush` take. We did not test whether width/height may differ from the uploaded PNG's dimensions; in all successful tests they matched.

3.2 display.setpixel() UNDOCUMENTED

```
display.setpixel(<image id>, <px>, <py>, <0xRRGGBB>)
```

Argument	Meaning
image id	Object id from the 7-argument <code>display.create()</code> above. Only an image object is accepted; every other object type is rejected with error 1717.
px, py	0-based coordinates within the image's pixel buffer. Valid range is 0...width-1 and 0...height-1.
colour	24-bit RGB packed as an integer, <code>0xRRGGBB</code> . <code>0x00FF00</code> is full-brightness green.

The write goes into the off-screen buffer. Nothing appears on the panel until you flush. There is no return value, and **no error is raised in Lua** on a bad call — check `eventlog`.

The name lives in the firmware's display-attribute table immediately next to `image`, which is what first suggested that an image object, rather than a screen, is the correct target. INFERRED

3.3 display.flush() UNDOCUMENTED

```
display.flush(<image id>)
```

Pushes the image's pixel buffer to the display processor. Call it **once per frame**, after a batch of `setpixel` calls — the paint is deferred, and the flush itself is too fast to measure (§5).

The argument is required and must be the *image* id:

- `display.flush(image_id)` — works.
- `display.flush(screen_id)` —
1717 Attribute flush does not apply to an object of type Custom Screen.
- `display.flush()` — -109 Missing parameter. Note that this *returns cleanly to Lua*; the failure is visible only in the event log. This is exactly the permissive-binding trap described in §1.

3.4 Checking whether a call worked

Because malformed calls do not raise, the reliable pattern while developing is to clear the event log, make the call, and read the log back:

```
eventlog.clear()
display.setpixel(im, 0, 0, 0xFF0000)
if eventlog.getcount() > 0 then
  print(eventlog.next())
else
  print("ACCEPTED")
end
```

4. Failure modes and error codes

These are the errors we actually triggered. They are more useful than the happy path, because each one identifies a specific way the call can be wrong.

What you did	What the event log says	Diagnosis
<code>setpixel</code> on an image created without trailing width/height (documented 5-arg form)	1130 Parameter x, expected value from 0 to -1	The bounds check computes $\text{max} = \text{width} - 1 = -1$, i.e. a zero-size pixel buffer. The object id itself is valid — you get 1130, not 1717.
Same, but the create call had 6 arguments instead of 7 (width given, height missing)	1130 Parameter y, expected value from 0 to -1	Width was accepted, height defaulted to zero. Confirms both trailing arguments are consumed as buffer dimensions.
<code>setpixel</code> on a user screen id	1717 Attribute pixel, does not apply to an object of type Custom Screen	Wrong target type. <code>pixel</code> is an image attribute.
<code>setpixel</code> on the home screen	1717 ... object of type Home Screen	Same.
<code>setpixel</code> on the graph screen	1717 ... object of type Graph Screen	Same. You cannot draw pixels onto the built-in graph.
Arguments shifted — a coordinate where the object id belongs	1703 Object id is invalid	Useful discriminator: 1703 means "not an object", 1717 means "wrong kind of object", 1130 means "right object, empty buffer".
<code>display.flush()</code> with no argument	-109 Missing parameter	The id is mandatory. No Lua error is raised.
<code>display.flush(screen_id)</code>	1717 Attribute flush does not apply to an object of type Custom Screen	Flush is per-image, not per-screen.
Any display setter with a wrong argument <i>count</i>	— nothing in Lua —	Permissive C binding: no error is raised, so signatures cannot be discovered with <code>pcall()</code> . Check the event log or the screen.
Junk arguments to display setters	— varies —	Can leave the physical panel fully dark. Recover with the sequence in §8; no power cycle needed.

5. Measured performance

Timed on the instrument with `timer.cleartime()` / `timer.gettime()`, so these include Lua interpreter overhead — which is the number that matters, because that is how you will call it.

Operation	Measured	Notes
<code>display.setpixel</code> , per pixel	10.9 μs	600-pixel row written in 0.0066 s.
600-column trace, including the arithmetic to compute each y	0.0127 s	So the per-column cost roughly doubles once you also compute the value — the interpreter, not the display link, is the limit.
<code>display.flush(im)</code>	0.0000 s	Too fast to measure. The paint is deferred, so batch and flush once per frame.
<code>buffer.write.reading()</code> into a <code>WRITABLE_FULL</code> buffer, plotted on the built-in graph (the documented path)	$\sim 80 \mu$ s	Per point. This is the baseline pixel writes are competing with.

Pixel writes are roughly 7 $\times$ faster than the documented plotting path. In practical terms, for a 750-point trace:

Path	750-point trace
<code>display.setpixel</code>	~ 8 ms
<code>buffer.write.reading()</code> + graph	~ 60 ms

Two consequences worth internalising:

- **Flush once per frame, never per pixel.** The whole speed advantage comes from batching into the off-screen buffer.
- **A full-canvas repaint is not affordable.** At 10.9 μ s per pixel, clearing a 750 $\times$ 500 canvas is 375 000 writes $\approx$ 4 s. That dictates the technique in §6.

6. Animation: differential redraw

Do not repaint the canvas per frame. Keep the previous y for each column and erase only that one pixel before drawing the new one. A frame then costs about `2 $\times$ W` writes instead of `W $\times$ H`.

```

local prev = {}
for x = 0, W-1 do prev[x] = -1 end

-- per frame:
for x = 0, W-1 do
  local y = compute(x)
  local p = prev[x]
  if p ~= y then
    if p >= 0 then display.setpixel(im, x, p, BG) end
    display.setpixel(im, x, y, FG)
    prev[x] = y
  end
end
display.flush(im)

```

Points that matter in the loop above:

- The sentinel `-1` marks "nothing drawn in this column yet", so the first frame skips the erase.

- The `p ~= y` test skips columns whose value did not move — free, and on a slow-moving trace it cuts the write count substantially.
- Hoist the function lookups out of the loop. In Lua 5.0.2 each `display.setpixel` is a metatable index; caching it in a local (`local sp = display.setpixel`, along with `math.floor`, `math.sin`) removes that per-write cost.
- For a 2-pixel-thick trace, erase and draw both rows: up to four writes per changed column, so ~3000 writes for a 750-wide frame. At the measured 10.9 μ s that is on the order of 30 ms of pixel writes per frame, versus 4 s for a full clear. **INFERRED** (arithmetic from the measured per-pixel cost, not a separately measured frame time.)

7. Practical display notes

- **Panel is 800×480.** Under an app's title bar the usable area is roughly **800×430**.
- **~750 px wide is a sensible canvas** for a plot that also needs room for axis legends.
- **Draw traces 2 px thick in y.** The physical pixels are very small; a 1-px trace is genuinely hard to see on the glass.
- **Use full-brightness colours on a near-black background.** `0x00FF00` on `0x000008` reads well. Mid-grey backgrounds wash the trace out badly — this was a real, visible failure in the first attempt, not a matter of taste.
- The pixel buffer's bounds come from the `create` call's width and height, not from what fits on screen. A 750×500 canvas placed at `y = 55` on a 480-pixel-tall panel accepted writes across its full height with no bounds errors; what is visible is limited by the panel.

8. Recovering a blacked-out screen

Bad arguments to display setters — and the undocumented `display.STATE_BLACKOUT` / `display.STATE_LCD_OFF` states — can leave the panel completely dark. You do not need a power cycle:

```
display.lightstate = display.STATE_LCD_100
display.changescreen(display.SCREEN_HOME)
```

Worth putting at the top of every experiment script, together with `reset()` and `eventlog.clear()`.

9. Things that do not exist

HALLUCINATION WARNING

Every name in this section is suggested confidently by AI assistants and web search results for "DMM6500 draw graphics". **We tested each one on the instrument. They are all nil or absent.** If a tool hands you code using them, the code has never run.

Suggested name	Reality
display.canvas	<code>nil</code> . There is no canvas object at all.
display.canvas.setpixel display.canvas.drawline display.canvas.drawrect	Do not exist — they cannot, since <code>display.canvas</code> is <code>nil</code> .
<code>_G.gui</code>	Absent. No such global.
drawline / drawrect / drawtext	No drawing primitives of any kind. A scan of all 130 <code>_G</code> entries found no key containing "draw", "canvas", "pixel" or "gfx" .
display.getpixel	The string is present in the firmware's string table but it is not callable : <code>attempt to call field 'getpixel' (a nil value)</code> . The pixel buffer is write-only from TSP.
buffer.write.readings buffer.write.batch buffer.write.block	All <code>nil</code> . There is no bulk or vectorised buffer write; <code>buffer.write.reading()</code> one point at a time is the only path.
display.graph.adddata display.graph.setdata display.graph.plot display.graph.append	All <code>nil</code> . You cannot push a data series at the built-in graph.

For lines, use what does exist: documented `display.OBJ_LINE` objects, or rasterise the line yourself pixel by pixel into an image canvas.

9.1 Object types named in firmware but unreachable

The firmware's internal object-type table names types with no public constants: `WINDOW`, `FAST_TEXT`, `STATIC_LABEL`, `SEP`, `ITEM_LIST`. Probing `display.OBJ_WINDOW`, `display.OBJ_FAST_TEXT` and the rest returns `nil`, so they are not reachable from TSP in this firmware. `OBJ_IMAGE` remains the only bitmap-ish type available.

10. Other undocumented names in the same firmware

The `display`, `display.graph` and `buffer` tables are metatable proxies: `pairs()` over them yields nothing, and the real API surface lives in the metatable. That is how the names below were found.

```
local mt = getmetatable(display)
-- mt.Objects (244 entries), mt.Getters (5), mt.Setters (4),
-- mt.__index, mt.__newindex
```


Name	Notes
<code>display.setpixel</code> <code>display.flush</code>	The subject of this document.
<code>display.setkeyenable</code>	Front-panel key enable/disable. Not exercised here.
<code>display.SCREEN_DATASHEET</code> <code>display.SCREEN_PLOT_SWIPE</code>	Built-in screens absent from the docs.
<code>display.OBJ_EDIT_NOTIFY_EVENT</code> <code>display.OBJ_EDIT_WAIT_EVENT</code>	Undocumented object types that do exist as constants.
<code>display.EVENT_SHOW</code> <code>display.EVENT_HIDE</code>	Undocumented event constants.
<code>display.STATE_BLACKOUT</code> <code>display.STATE_LCD_OFF</code>	Can turn the panel fully dark. See §8 before trying them.
<code>display.YSCALE_PER_TRACE_ONCE</code> <code>display.XSCALE_SHARED</code>	Extra graph scaling modes.
<code>display.TYPE_VIRTUAL</code>	Display-type value (virtual front panel vs. physical screen).

Existence here means the name resolves — it does not mean we characterised the call. Only `setpixel`, `flush`, the image `create` form and `STATE_LCD_100` were exercised end to end.

11. Environment gotchas

11.1 The Lua is 5.0.2

Older than nearly every Lua reference and every LLM's default assumption:

- **No `#` length operator.** Use `table.getn(t)`.
- **No bitwise operators.** There is a `bit` table with `bitand`, `bitor`, `bitxor`, `test`, `set`, `clear`, `get`, `getfield`, `setfield`. For packing a colour, plain arithmetic works: `r*65536 + g*256 + b`.
- The instrument is in **TSP mode**, so SCPI syntax is a syntax error, not a command.

11.2 One controlling client, and dead sockets

THIS WILL WASTE AN HOUR IF YOU DO NOT KNOW IT

Only one client may control the instrument over TCP at a time. A second concurrent socket does *not* error — it silently steals replies, so your reads return empty strings or the previous command's answer.

A client killed mid-session leaves a **dead socket** that swallows all responses. To clear every connection without a power cycle, connect to port **5030** (Dead Socket Termination) and immediately close it.

Port	Service
23	Telnet
80	HTTP (LXI web interface)
1024	VXI-11
5025	Raw socket — what you send TSP to
5030	Dead Socket Termination — connect and close to reset all connections

Reverse-engineered empirically on a Keithley DMM6500, firmware 1.7.17a. Unofficial and unsupported: nothing here is promised by Keithley, and a firmware update may change or remove any of it. Reproduce the measurements before relying on them.